KODE SUMBER PROGRAM KOMPUTER

Halaman awal dan halaman akhir

Sistem Promosi Digital Terintegrasi Berbasis SEO dan Machine Learning untuk Prediksi Perilaku Calon Peserta Pelatihan

Versi 1.0 — Pemegang Hak Cipta: Politeknik Negeri Medan

Dokumen ini memuat **10 halaman pertama** dan **10 halaman terakhir** dari listing kode sumber lengkap sebanyak **56 halaman** (24 berkas, 3.401 baris). Nomor halaman pada bagian bawah setiap halaman merujuk pada nomor halaman listing lengkap. Urutan berkas sesuai tabel berikut.

No	Berkas	Fungsi
1	`backend/app/main.py`	Titik masuk aplikasi FastAPI
2	`backend/app/config.py`	Konfigurasi aplikasi
3	`backend/app/db.py`	Koneksi basis data
4	`backend/app/models.py`	Skema basis data (ORM)
5	`backend/app/security.py`	Hash password & token login
6	`backend/app/traffic.py`	Klasifikasi sumber traffic & intent
7	`backend/app/features.py`	Feature engineering perilaku
8	`backend/app/ml_engine.py`	Pelatihan & perbandingan algoritma ML
9	`backend/app/scoring.py`	Skor konversi & segmentasi
10	`backend/app/recommend.py`	Rekomendasi konten
11	`backend/app/routers/public.py`	API untuk website (tracking, lead, rekomendasi)
12	`backend/app/routers/admin.py`	API dashboard admin
13	`backend/ml/train.py`	Perintah pelatihan model
14	`backend/jobs/gsc_etl.py`	ETL Google Search Console
15	`backend/jobs/scheduler.py`	Penjadwal tugas rutin
16	`backend/scripts/create_admin.py`	Pembuatan akun admin
17	`backend/app/static/tracker.js`	Script pelacak di website
18	`backend/app/static/dashboard/index.html`	Struktur halaman dashboard
19	`backend/app/static/dashboard/style.css`	Tampilan dashboard
20	`backend/app/static/dashboard/app.js`	Logika dashboard admin
21	`integration/wordpress/ysii-tracker/ysii-tracker.php`	Plugin WordPress
22	`backend/Dockerfile`	Image kontainer aplikasi
23	`docker-compose.yml`	Orkestrasi layanan
24	`nginx/templates/default.conf.template`	Konfigurasi server web

backend/app/main.py — Titik masuk aplikasi FastAPI (50 baris)

```
import logging
from contextlib import asynccontextmanager
from pathlib import Path

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import RedirectResponse
from fastapi.staticfiles import StaticFiles

from .config import get_settings
from .db import init_db
from .routers import admin, public

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(name)s: %(message)s")
STATIC_DIR = Path(__file__).parent / "static"


@asynccontextmanager
async def lifespan(_: FastAPI):
    init_db()
    yield


app = FastAPI(
    title="YSII — Sistem Promosi Digital Terintegrasi SEO & Machine Learning",
    description=(
        "API untuk tracking perilaku pengunjung, pendaftaran lead, prediksi konversi, "
        "rekomendasi konten, dan dashboard manajemen lead."
    ),
    version="1.0.0",
    lifespan=lifespan,
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=get_settings().origins,
    allow_methods=["GET", "POST"],
    allow_headers=["Content-Type"],
)

app.include_router(public.router)
app.include_router(admin.auth_router)
app.include_router(admin.router)
app.mount("/static", StaticFiles(directory=STATIC_DIR), name="static")
app.mount("/dashboard", StaticFiles(directory=STATIC_DIR / "dashboard", html=True), name="dashboard")


@app.get("/", include_in_schema=False)
def root():
    return RedirectResponse("/dashboard/")
```

backend/app/config.py — Konfigurasi aplikasi (46 baris)

```
from functools import lru_cache
from pathlib import Path

from pydantic_settings import BaseSettings, SettingsConfigDict

BASE_DIR = Path(__file__).resolve().parent.parent


class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

```

```
    database_url: str = f"sqlite:///{BASE_DIR / 'ysii.db'}"
    secret_key: str = "ganti-dengan-string-acak-panjang"
    token_expire_hours: int = 12

    # Domain website mitra yang boleh mengirim data tracking (dipisah koma)
    allowed_origins: str = "https://ysii.org,https://www.ysii.org"
    # URL publik API ini (dipakai tracker untuk memanggil balik)
    public_api_url: str = "https://api.ysii.org"
    # Nomor WhatsApp admin (format 62xxxx) untuk tombol "Hubungi" di dashboard
    whatsapp_admin: str = ""

    # Batas segmen skor konversi
    hot_threshold: float = 0.7
    warm_threshold: float = 0.4

    # Model ML
    model_dir: Path = BASE_DIR / "models"
    # Jumlah minimal contoh positif sebelum model ML menggantikan skor berbasis aturan
    min_positive_samples: int = 30

    # Google Search Console (opsional)
    gsc_site_url: str = "sc-domain:ysii.org"
    gsc_credentials_file: str = ""

    # Notifikasi lead "hot" (opsional, via webhook mis. n8n / Fonnte / Slack)
    hot_lead_webhook_url: str = ""

    @property
    def origins(self) -> list[str]:
        return [o.strip() for o in self.allowed_origins.split(",") if o.strip()]


@lru_cache
def get_settings() -> Settings:
    return Settings()
```

backend/app/db.py — Koneksi basis data (30 baris)

```
from collections.abc import Iterator

from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker

from .config import get_settings


class Base(DeclarativeBase):
    pass


_settings = get_settings()
_connect_args = {"check_same_thread": False} if _settings.database_url.startswith("sqlite") el
se {}
engine = create_engine(_settings.database_url, connect_args=_connect_args, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)


def get_db() -> Iterator[Session]:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()


def init_db() -> None:
    from . import models  # noqa: F401  (registrasi tabel)

```

```
    Base.metadata.create_all(bind=engine)
```

backend/app/models.py — Skema basis data (ORM) (171 baris)

```
"""Skema database (sesuai ERD di docs/LANGKAH-PEMBANGUNAN-SISTEM.md)."""

from datetime import date, datetime, timezone

from sqlalchemy import (
    JSON,
    Boolean,
    Date,
    DateTime,
    Float,
    ForeignKey,
    Index,
    Integer,
    String,
    Text,
)
from sqlalchemy.orm import Mapped, mapped_column, relationship

from .db import Base


def utcnow() -> datetime:
    return datetime.now(timezone.utc)


class Visitor(Base):
    __tablename__ = "visitors"

    visitor_id: Mapped[str] = mapped_column(String(36), primary_key=True)
    # Kode pendek yang ikut di pesan WhatsApp, agar chat WA bisa dihubungkan ke pengunjung
    ref_code: Mapped[str] = mapped_column(String(8), index=True)
    first_seen_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    last_seen_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    first_source: Mapped[str | None] = mapped_column(String(64))
    first_medium: Mapped[str | None] = mapped_column(String(32))
    first_campaign: Mapped[str | None] = mapped_column(String(128))
    first_landing: Mapped[str | None] = mapped_column(Text)
    device_type: Mapped[str | None] = mapped_column(String(16))

    sessions: Mapped[list["VisitSession"]] = relationship(back_populates="visitor")


class VisitSession(Base):
    __tablename__ = "sessions"

    session_id: Mapped[str] = mapped_column(String(36), primary_key=True)
    visitor_id: Mapped[str] = mapped_column(ForeignKey("visitors.visitor_id"), index=True)
    started_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    ended_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    source: Mapped[str | None] = mapped_column(String(64))
    medium: Mapped[str | None] = mapped_column(String(32))
    campaign: Mapped[str | None] = mapped_column(String(128))
    utm_term: Mapped[str | None] = mapped_column(String(256))
    landing_page: Mapped[str | None] = mapped_column(Text)
    referrer: Mapped[str | None] = mapped_column(Text)
    page_count: Mapped[int] = mapped_column(Integer, default=0)
    active_seconds: Mapped[int] = mapped_column(Integer, default=0)
    max_scroll_pct: Mapped[int] = mapped_column(Integer, default=0)

    visitor: Mapped[Visitor] = relationship(back_populates="sessions")


class Event(Base):
    __tablename__ = "events"
```

```
    __table_args__ = (Index("ix_events_visitor_ts", "visitor_id", "ts"),)

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    session_id: Mapped[str] = mapped_column(ForeignKey("sessions.session_id"))
    visitor_id: Mapped[str] = mapped_column(ForeignKey("visitors.visitor_id"))
    ts: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    # page_view, scroll, heartbeat, click_cta, click_whatsapp, view_price,
    # download_brochure, form_start, form_submit, site_search
    type: Mapped[str] = mapped_column(String(32), index=True)
    page_url: Mapped[str | None] = mapped_column(Text)
    page_category: Mapped[str | None] = mapped_column(String(32))
    program_slug: Mapped[str | None] = mapped_column(String(128), index=True)
    value: Mapped[float | None] = mapped_column(Float)
    label: Mapped[str | None] = mapped_column(String(256))

    # Relasi dipakai agar SQLAlchemy menyimpan visitor & session sebelum event (urutan FK)
    session: Mapped["VisitSession"] = relationship()
    visitor: Mapped["Visitor"] = relationship()


class Program(Base):
    __tablename__ = "programs"

    slug: Mapped[str] = mapped_column(String(128), primary_key=True)
    name: Mapped[str] = mapped_column(String(256))
    category: Mapped[str | None] = mapped_column(String(128))
    price: Mapped[float | None] = mapped_column(Float)
    url: Mapped[str | None] = mapped_column(Text)
    next_batch: Mapped[date | None] = mapped_column(Date)
    active: Mapped[bool] = mapped_column(Boolean, default=True)


LEAD_STATUSES = ["baru", "dihubungi", "tertarik", "terdaftar", "bayar", "batal"]
CONVERTED_STATUSES = {"terdaftar", "bayar"}


class Lead(Base):
    __tablename__ = "leads"

    lead_id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    visitor_id: Mapped[str | None] = mapped_column(ForeignKey("visitors.visitor_id"), index=Tr
ue)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    channel: Mapped[str] = mapped_column(String(16), default="form")  # form / whatsapp
    name: Mapped[str | None] = mapped_column(String(256))
    phone: Mapped[str | None] = mapped_column(String(32))
    email: Mapped[str | None] = mapped_column(String(256))
    program_slug: Mapped[str | None] = mapped_column(String(128))
    message: Mapped[str | None] = mapped_column(Text)
    extra: Mapped[dict | None] = mapped_column(JSON)
    status: Mapped[str] = mapped_column(String(16), default="baru")
    notes: Mapped[str | None] = mapped_column(Text)
    # Label ML: True = menjadi peserta (terdaftar/bayar), False = batal, None = belum diketahu
i
    converted: Mapped[bool | None] = mapped_column(Boolean)
    converted_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
    score: Mapped[float | None] = mapped_column(Float)
    segment: Mapped[str | None] = mapped_column(String(8))
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)

    visitor: Mapped[Visitor | None] = relationship()


class Prediction(Base):
    __tablename__ = "predictions"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    visitor_id: Mapped[str] = mapped_column(String(36), index=True)
```

```
    lead_id: Mapped[int | None] = mapped_column(Integer)
    model_version: Mapped[str] = mapped_column(String(80))
    score: Mapped[float] = mapped_column(Float)
    segment: Mapped[str] = mapped_column(String(8))
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)


class GscDaily(Base):
    __tablename__ = "gsc_daily"

    date: Mapped[date] = mapped_column(Date, primary_key=True)
    page: Mapped[str] = mapped_column(String(512), primary_key=True)
    query: Mapped[str] = mapped_column(String(512), primary_key=True)
    clicks: Mapped[int] = mapped_column(Integer)
    impressions: Mapped[int] = mapped_column(Integer)
    ctr: Mapped[float] = mapped_column(Float)
    position: Mapped[float] = mapped_column(Float)


class ModelRegistry(Base):
    __tablename__ = "model_registry"

    version: Mapped[str] = mapped_column(String(80), primary_key=True)
    # "lead" (lead -> peserta) atau "visitor" (pengunjung -> lead)
    task: Mapped[str] = mapped_column(String(16), default="lead", index=True)
    algorithm: Mapped[str] = mapped_column(String(64))
    trained_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    n_samples: Mapped[int] = mapped_column(Integer)
    n_positive: Mapped[int] = mapped_column(Integer)
    metrics: Mapped[dict] = mapped_column(JSON)
    is_active: Mapped[bool] = mapped_column(Boolean, default=False)
    artifact_path: Mapped[str] = mapped_column(Text)


class AdminUser(Base):
    __tablename__ = "admin_users"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    username: Mapped[str] = mapped_column(String(64), unique=True)
    password_hash: Mapped[str] = mapped_column(String(256))
    is_active: Mapped[bool] = mapped_column(Boolean, default=True)
```

backend/app/security.py — Hash password & token login (63 baris)

```
import base64
import hashlib
import hmac
import secrets
from datetime import datetime, timedelta, timezone

import jwt
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy import select
from sqlalchemy.orm import Session

from .config import get_settings
from .db import get_db
from .models import AdminUser

_ITERATIONS = 310_000
_bearer = HTTPBearer(auto_error=False)


def hash_password(password: str) -> str:
    salt = secrets.token_bytes(16)
    digest = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, _ITERATIONS)
    return "pbkdf2_sha256${}${}${}".format(
```

```
        _ITERATIONS, base64.b64encode(salt).decode(), base64.b64encode(digest).decode()
    )


def verify_password(password: str, stored: str) -> bool:
    try:
        _, iterations, salt_b64, digest_b64 = stored.split("$")
    except ValueError:
        return False
    digest = hashlib.pbkdf2_hmac(
        "sha256", password.encode(), base64.b64decode(salt_b64), int(iterations)
    )
    return hmac.compare_digest(digest, base64.b64decode(digest_b64))


def create_token(username: str) -> str:
    s = get_settings()
    payload = {
        "sub": username,
        "exp": datetime.now(timezone.utc) + timedelta(hours=s.token_expire_hours),
    }
    return jwt.encode(payload, s.secret_key, algorithm="HS256")


def current_admin(
    creds: HTTPAuthorizationCredentials | None = Depends(_bearer),
    db: Session = Depends(get_db),
) -> AdminUser:
    unauthorized = HTTPException(status.HTTP_401_UNAUTHORIZED, "Silakan login kembali")
    if creds is None:
        raise unauthorized
    try:
        payload = jwt.decode(creds.credentials, get_settings().secret_key, algorithms=["HS25
6"])
    except jwt.PyJWTError:
        raise unauthorized
    user = db.scalar(select(AdminUser).where(AdminUser.username == payload.get("sub")))
    if user is None or not user.is_active:
        raise unauthorized
    return user
```

backend/app/traffic.py — Klasifikasi sumber traffic & intent (96 baris)

```
"""Klasifikasi sumber traffic, kategori halaman, dan intent kata kunci."""

import re
from urllib.parse import urlparse

SEARCH_ENGINES = ("google.", "bing.", "yahoo.", "duckduckgo.", "yandex.", "ecosia.")
SOCIAL = {
    "facebook": ("facebook.", "fb.", "m.facebook"),
    "instagram": ("instagram.",),
    "tiktok": ("tiktok.",),
    "youtube": ("youtube.", "youtu.be"),
    "linkedin": ("linkedin.", "lnkd.in"),
    "twitter": ("twitter.", "t.co", "x.com"),
    "whatsapp": ("whatsapp.", "wa.me"),
    "telegram": ("t.me", "telegram."),
}
PAID_MEDIUMS = {"cpc", "ppc", "paid", "paidsearch", "paid_social", "ads"}


def classify_source(
    referrer: str | None,
    utm_source: str | None,
    utm_medium: str | None,
    own_hosts: tuple[str, ...] = ("ysii.org",),
) -> tuple[str, str]:
```

```
    """Kembalikan (source, medium), misal ('google', 'organic')."""
    if utm_source:
        medium = (utm_medium or "referral").lower()
        if medium in PAID_MEDIUMS:
            medium = "cpc"
        return utm_source.lower()[:64], medium[:32]

    host = (urlparse(referrer).hostname or "").lower() if referrer else ""
    if not host or any(host.endswith(h) for h in own_hosts):
        return "direct", "none"
    for engine in SEARCH_ENGINES:
        if engine in host:
            return engine.rstrip("."), "organic"
    for name, patterns in SOCIAL.items():
        if any(p in host for p in patterns):
            return name, "social"
    return host[:64], "referral"


def page_category(path: str | None, program_slug: str | None = None) -> str:
    if program_slug:
        return "program"
    p = (path or "/").lower()
    if p in ("", "/", "/index.html", "/home"):
        return "beranda"
    if re.search(r"/(program|pelatihan|kursus|training|sertifikasi|skema|kelas)", p):
        return "program"
    if re.search(r"/(daftar|pendaftaran|register|registrasi)", p):
        return "pendaftaran"
    if re.search(r"/(kontak|contact|hubungi)", p):
        return "kontak"
    if re.search(r"/(tentang|about|profil|visi)", p):
        return "tentang"
    if re.search(r"/(blog|artikel|berita|news|\d{4}/\d{2})", p):
        return "artikel"
    return "lainnya"


_INTENT_RULES = [
    ("brand", r"\b(ysii|sinergi inspirasi)\b"),
    ("transaksional", r"\b(daftar|pendaftaran|biaya|harga|jadwal|registrasi|promo|diskon)\b"),
    ("lokal", r"\b(medan|sumut|sumatera utara|terdekat|dekat)\b"),
    ("komersial", r"\b(pelatihan|kursus|sertifikasi|training|bnsp|kelas|bersertifikat|lsp)
\b"),
    ("informasional", r"\b(apa|cara|adalah|bagaimana|manfaat|contoh|pengertian|tips|prospek)
\b"),
]


def query_intent(query: str | None) -> str:
    """Kategori intent kata kunci pencarian (aturan sederhana, bisa diperluas)."""
    if not query:
        return "tidak_diketahui"
    q = query.lower()
    for intent, pattern in _INTENT_RULES:
        if re.search(pattern, q):
            return intent
    return "lainnya"


def normalize_path(url: str | None) -> str:
    if not url:
        return "/"
    parsed = urlparse(url)
    return (parsed.path or "/").rstrip("/") or "/"


```

```
_PROGRAM_PATH = re.compile(r"/(?:program|pelatihan|kursus|sertifikasi|training|kelas)/([a-z0-9-]+)", re.I)


def program_from_path(path: str | None) -> str | None:
    m = _PROGRAM_PATH.search(path or "")
    return m.group(1).lower() if m else None
```

backend/app/features.py — Feature engineering perilaku (190 baris)

```
"""Feature engineering perilaku pengunjung (dipakai bersama oleh training & scoring).

Satu baris fitur = satu pengunjung, dihitung dari event-nya sampai waktu `as_of`.
Untuk data latih, `as_of` = saat pengunjung menjadi lead, supaya model hanya belajar
dari perilaku SEBELUM mendaftar (mencegah kebocoran label).
"""

from collections import defaultdict
from collections.abc import Iterable
from datetime import datetime, timezone

from sqlalchemy import func, select
from sqlalchemy.orm import Session

from .models import Event, GscDaily, Visitor
from .traffic import normalize_path, page_category, query_intent

NUMERIC_FEATURES = [
    "n_sessions",
    "n_pageviews",
    "total_active_sec",
    "avg_active_per_session",
    "avg_pages_per_session",
    "max_scroll_pct",
    "avg_session_scroll_pct",
    "bounce_rate",
    "n_program_views",
    "n_distinct_programs",
    "n_article_views",
    "n_deep_program_reads",
    "n_click_cta",
    "n_click_whatsapp",
    "n_view_price",
    "n_download_brochure",
    "n_form_start",
    "n_site_search",
    "is_returning",
    "n_active_days",
    "engagement_span_days",
    "first_visit_hour",
    "first_visit_weekend",
]
CATEGORICAL_FEATURES = [
    "first_medium",
    "first_source",
    "device_type",
    "landing_category",
    "landing_query_intent",
]
ALL_FEATURES = NUMERIC_FEATURES + CATEGORICAL_FEATURES

# Fitur yang dikeluarkan per tugas. Untuk "pengunjung -> lead", klik WhatsApp dan mulai
# mengisi form adalah aksi menjadi lead itu sendiri, jadi memakainya = membocorkan label.
_EXCLUDED = {"visitor": {"n_click_whatsapp", "n_form_start"}, "lead": set()}


def task_features(task: str) -> tuple[list[str], list[str]]:
    excluded = _EXCLUDED[task]
```

```
    return (
        [f for f in NUMERIC_FEATURES if f not in excluded],
        [f for f in CATEGORICAL_FEATURES if f not in excluded],
    )

_COUNTED_EVENTS = {
    "click_cta": "n_click_cta",
    "click_whatsapp": "n_click_whatsapp",
    "view_price": "n_view_price",
    "download_brochure": "n_download_brochure",
    "form_start": "n_form_start",
    "site_search": "n_site_search",
}
_TOP_SOURCES = {"google", "direct", "facebook", "instagram", "tiktok", "youtube", "whatsapp"}


def _aware(dt: datetime) -> datetime:
    return dt if dt.tzinfo else dt.replace(tzinfo=timezone.utc)


def compute_features(
    visitor: Visitor,
    events: Iterable[Event],
    landing_intents: dict[str, str],
    as_of: datetime | None = None,
) -> dict:
    cutoff = _aware(as_of) if as_of else None
    evs = sorted(
        (e for e in events if cutoff is None or _aware(e.ts) <= cutoff),
        key=lambda e: _aware(e.ts),
    )

    f: dict = {k: 0 for k in NUMERIC_FEATURES}
    per_session_pages: dict[str, int] = defaultdict(int)
    per_session_active: dict[str, float] = defaultdict(float)
    per_session_scroll: dict[str, float] = defaultdict(float)
    programs: set[str] = set()
    days: set = set()

    for e in evs:
        sid = e.session_id
        per_session_pages.setdefault(sid, 0)
        days.add(_aware(e.ts).date())
        if e.type == "page_view":
            per_session_pages[sid] += 1
            f["n_pageviews"] += 1
            if e.page_category == "program":
                f["n_program_views"] += 1
                if e.program_slug:
                    programs.add(e.program_slug)
            elif e.page_category == "artikel":
                f["n_article_views"] += 1
        elif e.type == "heartbeat":
            per_session_active[sid] += max(0.0, min(float(e.value or 0), 1800.0))
        elif e.type == "scroll":
            v = float(e.value or 0)
            per_session_scroll[sid] = max(per_session_scroll[sid], v)
            if v >= 75 and e.page_category == "program":
                f["n_deep_program_reads"] += 1
        elif e.type in _COUNTED_EVENTS:
            f[_COUNTED_EVENTS[e.type]] += 1

    sessions = list(per_session_pages)
    n_sessions = len(sessions)
    f["n_sessions"] = n_sessions
    f["n_distinct_programs"] = len(programs)
    f["total_active_sec"] = round(sum(per_session_active.values()), 1)
    if n_sessions:
```

```
        f["avg_active_per_session"] = round(f["total_active_sec"] / n_sessions, 1)
        f["avg_pages_per_session"] = round(f["n_pageviews"] / n_sessions, 2)
        f["max_scroll_pct"] = max((per_session_scroll.get(s, 0) for s in sessions), default=0)
        f["avg_session_scroll_pct"] = round(
            sum(per_session_scroll.get(s, 0) for s in sessions) / n_sessions, 1
        )
        bounces = sum(
            1
            for s in sessions
            if per_session_pages[s] <= 1 and per_session_active.get(s, 0) < 10
        )
        f["bounce_rate"] = round(bounces / n_sessions, 3)
    f["is_returning"] = int(n_sessions > 1)
    f["n_active_days"] = len(days)

    # Rentang hari dari kunjungan pertama sampai aktivitas terakhir (sebelum cutoff).
    # Sengaja TIDAK memakai "hari sejak kunjungan terakhir sampai sekarang": nilai itu
    # berbeda cara hitungnya antara lead (dipotong saat mendaftar) dan non-lead,
    # sehingga membocorkan label ke model.
    first = _aware(visitor.first_seen_at)
    last_ts = _aware(evs[-1].ts) if evs else first
    f["engagement_span_days"] = round(max(0.0, (last_ts - first).total_seconds() / 86400), 2)
    f["first_visit_hour"] = first.hour
    f["first_visit_weekend"] = int(first.weekday() >= 5)

    source = (visitor.first_source or "direct").lower()
    landing_path = normalize_path(visitor.first_landing)
    f["first_medium"] = visitor.first_medium or "none"
    f["first_source"] = source if source in _TOP_SOURCES else "lainnya"
    f["device_type"] = visitor.device_type or "unknown"
    f["landing_category"] = page_category(landing_path)
    f["landing_query_intent"] = landing_intents.get(landing_path, "tidak_diketahui")
    return f


def landing_intents(db: Session, paths: Iterable[str] | None = None) -> dict[str, str]:
    """Intent kata kunci dominan (berdasar klik GSC) untuk setiap landing page.

    Google tidak memberikan keyword per pengunjung, jadi keyword dominan dari
    halaman pendaratan dipakai sebagai proksi intent pencarian.
    """
    stmt = select(GscDaily.page, GscDaily.query, func.sum(GscDaily.clicks)).group_by(
        GscDaily.page, GscDaily.query
    )
    best: dict[str, tuple[str, int]] = {}
    wanted = set(paths) if paths is not None else None
    for page, query, clicks in db.execute(stmt):
        path = normalize_path(page)
        if wanted is not None and path not in wanted:
            continue
        if path not in best or (clicks or 0) > best[path][1]:
            best[path] = (query, clicks or 0)
    return {p: query_intent(q) for p, (q, _) in best.items()}


def visitor_features(
    db: Session, visitor_id: str, as_of: datetime | None = None
) -> dict | None:
    visitor = db.get(Visitor, visitor_id)
    if visitor is None:
        return None
    events = db.scalars(select(Event).where(Event.visitor_id == visitor_id)).all()
    intents = landing_intents(db, [normalize_path(visitor.first_landing)])
    return compute_features(visitor, events, intents, as_of=as_of)
```

backend/app/ml_engine.py — Pelatihan & perbandingan algoritma ML (287 baris)

```
"""ML Predictive Engine — pelatihan (CRISP-DM) dan perbandingan tiga algoritma.
```

Halaman 11 – 46 tidak ditampilkan

Bagian berikutnya adalah halaman terakhir listing kode sumber.

```
    data: {
      labels: d.series.map((s) => s.date.slice(5)),
      datasets: [
        { label: "Sesi", data: d.series.map((s) => s.sessions), borderColor: css("--cold"), tension: 0.3, yAxisID: "y" },
        { label: "Sesi organik", data: d.series.map((s) => s.organic), borderColor: css("--primary"), tension: 0.3, yAxisID: "y" },
        { label: "Lead", data: d.series.map((s) => s.leads), borderColor: css("--hot"), type: "bar", backgroundColor: css("--hot"), yAxisID: "y1" },
      ],
    },
    options: { interaction: { mode: "index", intersect: false }, scales: { y: { beginAtZero: true }, y1: { beginAtZero: true, position: "right", grid: { drawOnChartArea: false } } } },
  });
  const seg = ["hot", "warm", "cold"];
  chart("chart-segments", {
    type: "bar",
    data: { labels: ["Hot", "Warm", "Cold"], datasets: [{ label: "Lead", data: seg.map((s) => d.segments[s] || 0), backgroundColor: seg.map((s) => css(`--${s}`)) }] },
    options: { plugins: { legend: { display: false } }, scales: { y: { beginAtZero: true, ticks: { precision: 0 } } } },
  });
  $("#sources-table").innerHTML = "<tr><th>Sumber / medium</th><th class='num'>Sesi</th></tr>" +
    (d.sources.map(([s, n]) => `<tr><td>${esc(s)}</td><td class="num">${n}</td></tr>`).join("") || "<tr><td colspan=2 class='muted'>Belum ada data</td></tr>");
  const m = d.model;
  $("#model-summary").innerHTML = m.metrics
    ? `<dl class="kv"><dt>Algoritma</dt><dd>${esc(m.algorithm)}</dd><dt>Tugas</dt><dd>${esc(m.task_label)}</dd><dt>F1 (CV)</dt><dd>${m.metrics.f1}</dd><dt>AUC (CV)</dt><dd>${m.metrics.roc_auc ?? "-"}</dd><dt>Akurasi (CV)</dt><dd>${m.metrics.accuracy}</dd><dt>Dilatih</dt><dd>${fmtDate(m.trained_at)}</dd></dl>`
    : `<p>Memakai <b>skor berbasis aturan</b> karena data latih belum cukup. Model ML dapat dilatih di tab <a href="#model">Model ML</a>.</p>`;
}

// ---------------- lead ----------------
$('#lead-filter [name="status"]').innerHTML = `<option value="berjalan" selected>Masih berjalan</option><option value="">Semua status</option>${STATUSES.map((s) => `<option>${s}</option>`).join("")}`;
let filterTimer;
$("#lead-filter").addEventListener("input", () => { clearTimeout(filterTimer); filterTimer = setTimeout(() => { leadPage = 1; loadLeads(); }, 300); });
$("#lead-filter").addEventListener("submit", (e) => e.preventDefault());
$("#prev").addEventListener("click", () => { if (leadPage > 1) { leadPage--; loadLeads(); } });
$("#next").addEventListener("click", () => { leadPage++; loadLeads(); });

function waLink(phone, name) {
  if (!phone) return "";
  const text = encodeURIComponent(`Halo ${name || ""}, terima kasih sudah menghubungi Yayasan Sinergi Inspirasi Indonesia. `);
  return `<a class="btn small" target="_blank" rel="noopener" href="https://wa.me/${esc(phone)}?text=${text}">WhatsApp</a>`;
}
function scoreCell(l) {
  if (l.score == null) return "<span class='muted'>-</span>";
  return `<span class="badge ${esc(l.segment)}">${esc(l.segment)}</span> ${(l.score * 100).toFixed(0)}%<div class="bar"><i style="width:${(l.score * 100).toFixed(0)}%"></i></div>`;
}

async function loadLeads() {
  const params = new URLSearchParams(new FormData($("#lead-filter")));
  params.set("page", leadPage);
  params.set("limit", 25);
  const d = await api(`/v1/admin/leads?${params}`);
  const pages = Math.max(1, Math.ceil(d.total / d.limit));
  if (leadPage > pages) { leadPage = pages; return loadLeads(); }
```

```
  $("#lead-count").textContent = `${d.total} lead · halaman ${d.page} dari ${pages}`;
  $("#prev").disabled = leadPage <= 1;
  $("#next").disabled = leadPage >= pages;
  $("#lead-table").innerHTML =
    "<tr><th>Skor</th><th>Nama</th><th>Kontak</th><th>Program</th><th>Kanal</th><th>Masuk</th><th>Status</th><th></th></tr>" +
    (d.items.map((l) => `<tr class="click" data-id="${l.lead_id}">
      <td>${scoreCell(l)}</td><td>${esc(l.name || "-")}</td>
      <td>${esc(l.phone || "")}<br><span class="muted">${esc(l.email || "")}</span></td>
      <td>${esc(l.program_name || l.program_slug || "-")}</td><td>${esc(l.channel)}</td>
      <td>${fmtDate(l.created_at)}</td>
      <td><select data-status="${l.lead_id}" aria-label="Status">${STATUSES.map((s) => `<option ${s === l.status ? "selected" : ""}>${s}</option>`).join("")}</select></td>
      <td>${waLink(l.phone, l.name)}</td></tr>`).join("") || "<tr><td colspan=8 class='muted'>Belum ada lead</td></tr>");
}

$("#lead-table").addEventListener("change", async (e) => {
  const id = e.target.dataset.status;
  if (!id) return;
  try { await api(`/v1/admin/leads/${id}`, { method: "PATCH", body: JSON.stringify({ status: e.target.value }) }); }
  catch (err) { alert(err.message); }
});
$("#lead-table").addEventListener("click", (e) => {
  if (e.target.closest("select, a")) return;
  const tr = e.target.closest("tr[data-id]");
  if (tr) openLead(tr.dataset.id);
});

async function openLead(id) {
  const d = await api(`/v1/admin/leads/${id}`);
  const l = d.lead, v = d.visitor, s = d.score;
  const extra = l.extra ? Object.entries(l.extra).map(([k, val]) => `<dt>${esc(k)}</dt><dd>${esc(val)}</dd>`).join("") : "";
  $("#lead-detail").innerHTML = `
    <div class="row between"><h2>${esc(l.name || "Lead #" + l.lead_id)}</h2><button class="btn ghost" id="close-lead" type="button">Tutup</button></div>
    <div class="grid2">
      <div>
        <dl class="kv">
          <dt>Skor konversi</dt><dd>${scoreCell(l)}</dd>
          <dt>Kontak</dt><dd>${esc(l.phone || "-")} ${waLink(l.phone, l.name)}<br>${esc(l.email || "")}</dd>
          <dt>Program</dt><dd>${esc(l.program_name || l.program_slug || "-")}</dd>
          <dt>Kanal</dt><dd>${esc(l.channel)}</dd><dt>Masuk</dt><dd>${fmtDate(l.created_at)}</dd>
          <dt>Pesan</dt><dd>${esc(l.message || "-")}</dd>${extra}
          ${v ? `<dt>Sumber pertama</dt><dd>${esc(v.first_source)} / ${esc(v.first_medium)}</dd>
          <dt>Landing page</dt><dd>${esc(v.first_landing)}</dd><dt>Perangkat</dt><dd>${esc(v.device_type || "-")}</dd>
          <dt>Kode WA</dt><dd>${esc(v.ref_code)}</dd>` : "<dt>Riwayat</dt><dd class='muted'>Tidak terhubung dengan data kunjungan</dd>"}
        </dl>
        ${s ? `<h3 style="margin-top:12px">Faktor pendorong skor</h3><ul class="chips">${s.reasons.map((r) => `<li>${esc(r)}</li>`).join("") || "<li>-</li>"}</ul><p class="muted">Model: ${esc(s.model_version)}</p>` : ""}
        <form id="lead-edit" class="form-grid" style="margin-top:12px">
          <label>Status <select name="status">${STATUSES.map((x) => `<option ${x === l.status ? "selected" : ""}>${x}</option>`).join("")}</select></label>
          <label class="full">Catatan follow-up <textarea name="notes" rows="3">${esc(l.notes || "")}</textarea></label>
          <div class="row full end"><button class="btn ghost" type="button" id="rescore">Hitung ulang skor</button><button class="btn primary" type="submit">Simpan</button></div>
        </form>
      </div>
      <div>
```

```
        <h3>Riwayat kunjungan (${d.sessions.length} sesi)</h3>
        <table class="table"><tr><th>Waktu</th><th>Sumber</th><th class="num">Hal.</th><th cla
ss="num">Detik</th><th class="num">Scroll</th></tr>
          ${d.sessions.map((x) => `<tr><td>${fmtDate(x.started_at)}</td><td>${esc(x.source)}/
${esc(x.medium)}${x.utm_term ? `<br><span class="muted">${esc(x.utm_term)}</span>` : ""}</td><
td class="num">${x.page_count}</td><td class="num">${x.active_seconds}</td><td class="num">
${x.max_scroll_pct}%</td></tr>`).join("")}
        </table>
        <h3>Aktivitas</h3>
        <ul class="timeline">${d.timeline.map((t) => `<li><span class="t">${fmtDate(t.ts)}</sp
an>${esc(EVENT_TEXT[t.type] || t.type)} <span class="muted">${esc(t.label || t.page || "")}</s
pan></li>`).join("") || "<li class='muted'>-</li>"}</ul>
      </div>
    </div>`;
  const dlg = $("#lead-dialog");
  $("#close-lead").onclick = () => dlg.close();
  $("#rescore").onclick = async () => { await api(`/v1/admin/leads/${id}/rescore`, { method:
"POST" }); openLead(id); loadLeads(); };
  $("#lead-edit").onsubmit = async (e) => {
    e.preventDefault();
    await api(`/v1/admin/leads/${id}`, { method: "PATCH", body: JSON.stringify(Object.fromEntr
ies(new FormData(e.target))) });
    dlg.close();
    loadLeads();
  };
  if (!dlg.open) dlg.showModal();
}

$("#export-csv").addEventListener("click", async (e) => {
  e.preventDefault();
  const res = await fetch("/v1/admin/export/leads.csv", { headers: { Authorization: `Bearer
${token}` } });
  if (!res.ok) return alert("Gagal mengunduh");
  const a = document.createElement("a");
  a.href = URL.createObjectURL(await res.blob());
  a.download = `lead-ysii-${new Date().toISOString().slice(0, 10)}.csv`;
  a.click();
  URL.revokeObjectURL(a.href);
});

$("#btn-add-wa").addEventListener("click", () => { $("#wa-form").reset(); $("#wa-error").textC
ontent = ""; $("#wa-dialog").showModal(); });
$("#wa-submit").addEventListener("click", async (e) => {
  e.preventDefault();
  const form = $("#wa-form");
  if (!form.reportValidity()) return;
  const body = Object.fromEntries(new FormData(form));
  Object.keys(body).forEach((k) => body[k] === "" && delete body[k]);
  try {
    await api("/v1/admin/leads", { method: "POST", body: JSON.stringify(body) });
    $("#wa-dialog").close();
    loadLeads();
  } catch (err) { $("#wa-error").textContent = err.message; }
});

// ---------------- program ----------------
async function loadPrograms() {
  await refreshPrograms();
  $("#program-table").innerHTML = "<tr><th>Slug</th><th>Nama</th><th>Kategori</th><th class='n
um'>Harga</th><th>Batch</th><th>Aktif</th><th></th></tr>" +
    (programs.map((p) => `<tr><td><code>${esc(p.slug)}</code></td><td>${p.url ? `<a href="${es
c(p.url)}" target="_blank" rel="noopener">${esc(p.name)}</a>` : esc(p.name)}</td>
      <td>${esc(p.category || "-")}</td><td class="num">${p.price != null ? Number(p.price).to
LocaleString("id-ID") : "-"}</td>
      <td>${esc(p.next_batch || "-")}</td><td>${p.active ? "Ya" : "Tidak"}</td>
      <td class="row"><button class="btn small" data-edit="${esc(p.slug)}" type="button">Ubah
</button><button class="btn small" data-del="${esc(p.slug)}" type="button">Hapus</button></td>
</tr>`).join("") ||
```

```
      "<tr><td colspan=7 class='muted'>Belum ada program. Tambahkan di bawah.</td></tr>");
}
$("#program-table").addEventListener("click", async (e) => {
  const edit = e.target.dataset.edit, del = e.target.dataset.del;
  if (edit) {
    const p = programs.find((x) => x.slug === edit);
    const f = $("#program-form");
    ["slug", "name", "category", "price", "url", "next_batch"].forEach((k) => (f.elements[k].value = p[k] ?? ""));
    f.elements.active.checked = p.active;
    f.scrollIntoView({ behavior: "smooth" });
  }
  if (del && confirm(`Hapus program ${del}?`)) { await api(`/v1/admin/programs/${del}`, { method: "DELETE" }); loadPrograms(); }
});
$("#program-form").addEventListener("submit", async (e) => {
  e.preventDefault();
  const f = e.target;
  const body = {
    slug: f.elements.slug.value.trim(), name: f.elements.name.value.trim(), category: f.elements.category.value || null,
    price: f.elements.price.value ? Number(f.elements.price.value) : null, url: f.elements.url.value || null,
    next_batch: f.elements.next_batch.value || null, active: f.elements.active.checked,
  };
  try { await api(`/v1/admin/programs/${body.slug}`, { method: "PUT", body: JSON.stringify(body) }); f.reset(); loadPrograms(); }
  catch (err) { alert(err.message); }
});

// ---------------- SEO ----------------
async function loadSeo() {
  const d = await api("/v1/admin/seo?days=28");
  if (!d.available) {
    $("#seo-body").innerHTML = `<div class="card"><p>Belum ada data Google Search Console. Atur <code>GSC_CREDENTIALS_FILE</code> lalu jalankan
      <code>python -m jobs.gsc_etl --days 90</code> (lihat README).</p></div>`;
    return;
  }
  const rows = (list, key) => list.map((r) => `<tr><td>${esc(r[key])}</td><td class="num">${r.clicks}</td><td class="num">${r.impressions}</td><td class="num">${pct(r.ctr)}</td><td class="num">${r.position ?? "-"}</td></tr>`).join("");
  const head = (k) => `<tr><th>${k}</th><th class="num">Klik</th><th class="num">Impresi</th><th class="num">CTR</th><th class="num">Posisi</th></tr>`;
  $("#seo-body").innerHTML = `
    <div class="tiles">
      <div class="tile"><div class="v">${d.clicks.toLocaleString("id-ID")}</div><div class="l">Klik organik (28 hari)</div></div>
      <div class="tile"><div class="v">${d.impressions.toLocaleString("id-ID")}</div><div class="l">Impresi</div></div>
      <div class="tile"><div class="v">${pct(d.ctr)}</div><div class="l">CTR</div></div>
      <div class="tile"><div class="v">${d.avg_position ?? "-"}</div><div class="l">Rata-rata posisi</div></div>
    </div>
    <div class="card"><h3>Tren harian</h3><canvas id="chart-seo" height="90"></canvas></div>
    <div class="grid2">
      <div class="card"><h3>Kata kunci teratas</h3><table class="table">${head("Kata kunci")}${rows(d.top_queries, "query")}</table></div>
      <div class="card"><h3>Halaman teratas</h3><table class="table">${head("Halaman")}${rows(d.top_pages, "page")}</table></div>
    </div>
    ${d.low_ctr_pages.length ? `<div class="card"><h3>Perlu perbaikan judul/meta (impresi tinggi, CTR &lt; 2%)</h3><table class="table">${head("Halaman")}${rows(d.low_ctr_pages, "page")}</table></div>` : ""}`;
  chart("chart-seo", {
    type: "line",
    data: { labels: d.series.map((s) => s.date.slice(5)), datasets: [
```

```
      { label: "Klik", data: d.series.map((s) => s.clicks), borderColor: css("--primary"), yAxisID: "y", tension: 0.3 },
      { label: "Impresi", data: d.series.map((s) => s.impressions), borderColor: css("--cold"), yAxisID: "y1", tension: 0.3 },
    ] },
    options: { interaction: { mode: "index", intersect: false }, scales: { y: { beginAtZero: true }, y1: { beginAtZero: true, position: "right", grid: { drawOnChartArea: false } } } },
  });
}

// ---------------- model ----------------
let modelPoll;
const ALGO_NAME = { logistic_regression: "Logistic Regression (baseline)", random_forest: "Random Forest", gradient_boosting: "Gradient Boosting" };

function comparisonCard(m) {
  const r = m.report;
  const best = r.comparison[r.best_algorithm];
  return `<div class="card"><h3>${esc(r.task_label)} — model aktif <code>${esc(m.version)}</code></h3>
    <p class="muted">${r.n_samples} data (${r.n_positive} positif) · ${r.cv_folds}-fold stratified CV · data uji 20% · dilatih ${fmtDate(m.trained_at)}</p>
    <table class="table"><tr><th>Algoritma</th><th class="num">Akurasi</th><th class="num">Presisi</th><th class="num">Recall</th><th class="num">F1</th><th class="num">AUC</th><th class="num">F1 (uji)</th><th class="num">AUC (uji)</th></tr>
    ${Object.entries(r.comparison).map(([k, v]) => `<tr${k === r.best_algorithm ? ' style="font-weight:700"' : ""}><td>${esc(ALGO_NAME[k] || k)}${k === r.best_algorithm ? " ★" : ""}</td>
      <td class="num">${v.cv.accuracy}</td><td class="num">${v.cv.precision}</td><td class="num">${v.cv.recall}</td><td class="num">${v.cv.f1} ± ${v.cv.f1_std}</td><td class="num">${v.cv.roc_auc ?? "-"}</td>
      <td class="num">${v.holdout.f1}</td><td class="num">${v.holdout.roc_auc ?? "-"}</td></tr>`).join("")}</table>
    <p class="muted">Confusion matrix model terbaik pada data uji [[TN, FP], [FN, TP]]: ${esc(JSON.stringify(best.holdout.confusion_matrix))}</p>
    <h3>Kepentingan fitur (permutation importance)</h3>
    <table class="table"><tr><th>Fitur</th><th class="num">Penurunan AUC</th></tr>${r.feature_importance.slice(0, 10).map((f) => `<tr><td>${esc(f.feature)}</td><td class="num">${f.importance}</td></tr>`).join("")}</table>
    <button class="btn small" data-rules="${esc(m.task)}" type="button">Kembali ke skor aturan untuk tugas ini</button></div>`;
}

async function loadModel() {
  const d = await api("/v1/admin/model");
  const t = d.training;
  let status = "";
  if (t.running) status = "<p><b>Pelatihan sedang berjalan…</b></p>";
  else if (t.last_error) status = `<p class="error">${esc(t.last_error)}</p>`;
  else if (t.last_result) status = `<p>Hasil pelatihan terakhir:</p><ul>${Object.entries(t.last_result).map(([k, v]) => `<li><b>${esc(d.tasks[k])}</b>: ${esc(v)}</li>`).join("")}</ul>`;

  const activeCards = Object.keys(d.tasks).map((task) => {
    const m = d.models.find((x) => x.task === task && x.is_active);
    return m ? comparisonCard(m)
      : `<div class="card"><h3>${esc(d.tasks[task])}</h3><p>Memakai <b>skor berbasis aturan</b> (${esc(d.rule_version)}). Model ML aktif setelah ada minimal ${d.min_samples} contoh positif dan ${d.min_samples} negatif.</p></div>`;
  }).join("");

  $("#model-body").innerHTML = `
    <div class="card">${status}
      <p>Data berlabel: <b>${d.labeled.leads}</b> lead (${d.labeled.linked} terhubung dengan data kunjungan), <b>${d.labeled.converted}</b> menjadi peserta, <b>${d.labeled.batal}</b> batal.
      Status lead yang diisi tim di tab Lead menjadi data latih baru setiap kali model dilatih ulang (otomatis setiap Senin 02.00 WIB).</p>
    </div>
    ${activeCards}
```

```
    <div class="card"><h3>Riwayat model</h3><table class="table"><tr><th>Versi</th><th>Tugas</th><th>Dilatih</th><th class="num">Data</th><th class="num">Positif</th><th class="num">F1 CV</th><th class="num">AUC CV</th><th></th></tr>
      ${d.models.map((m) => { const cv = m.report.comparison[m.algorithm].cv; return `<tr><td><code>${esc(m.version)}</code></td><td>${esc(m.task)}</td><td>${fmtDate(m.trained_at)}</td><td class="num">${m.n_samples}</td><td class="num">${m.n_positive}</td>
        <td class="num">${cv.f1}</td><td class="num">${cv.roc_auc ?? "-"}</td><td>${m.is_active ? "<b>aktif</b>" : `<button class="btn small" data-activate="${esc(m.version)}" type="button">Aktifkan</button>`}</td></tr>`; }).join("") || "<tr><td colspan=8 class='muted'>Belum ada model</td></tr>"}
    </table></div>`;
  clearTimeout(modelPoll);
  if (t.running) modelPoll = setTimeout(loadModel, 3000);
}
$("#model-body").addEventListener("click", async (e) => {
  const v = e.target.dataset.activate, rules = e.target.dataset.rules;
  if (v) { await api(`/v1/admin/model/${encodeURIComponent(v)}/activate`, { method: "POST" }); loadModel(); }
  if (rules) { await api(`/v1/admin/model/deactivate?task=${encodeURIComponent(rules)}`, { method: "POST" }); loadModel(); }
});
$("#btn-retrain").addEventListener("click", async () => {
  try {
    await api("/v1/admin/model/retrain", { method: "POST", body: JSON.stringify({ task: $("#retrain-task").value, force: $("#retrain-force").checked }) });
    setTimeout(loadModel, 500);
  } catch (err) { alert(err.message); }
});

// ---------------- start ----------------
if (token) boot(); else logout();
```

integration/wordpress/ysii-tracker/ysii-tracker.php — Plugin WordPress (113 baris)

```
<?php
/**
 * Plugin Name: YSII Tracker (SEO & ML)
 * Description: Menghubungkan website dengan Sistem Promosi Digital Terintegrasi SEO & Machine Learning: tracking perilaku (dengan persetujuan), kode referensi WhatsApp, salinan form pendaftaran, dan widget rekomendasi [ysii_reco].
 * Version: 1.0.0
 * Author: Tim PTI Politeknik Negeri Medan
 * License: GPLv2 or later
 */

if (!defined('ABSPATH')) {
    exit;
}

const YSII_OPTION = 'ysii_tracker_settings';

function ysii_settings() {
    return wp_parse_args(get_option(YSII_OPTION, []), [
        'api_url'      => 'https://api.ysii.org',
        'privacy_url'  => home_url('/kebijakan-privasi/'),
        'consent'      => 'required',
        'mirror_forms' => 'auto',
        'whatsapp'     => '',
        'daftar_url'   => home_url('/pendaftaran/'),
    ]);
}

// 1) Pasang tracker.js di <head>
add_action('wp_head', function () {
    if (is_admin() || is_user_logged_in() && current_user_can('edit_posts')) {
        return; // jangan rekam admin/editor yang sedang mengelola situs
    }
    $s = ysii_settings();
```

```
    printf(
        '<script src="%s" defer data-privacy-url="%s" data-consent="%s" data-mirror-forms="%
s"></script>' . "\n",
        esc_url(rtrim($s['api_url'], '/') . '/static/tracker.js'),
        esc_url($s['privacy_url']),
        esc_attr($s['consent']),
        esc_attr($s['mirror_forms'])
    );
    // Tandai halaman program agar slug-nya terbaca (meta "ysii:program" diisi dari custom fie
ld)
    if (is_singular()) {
        $slug = get_post_meta(get_the_ID(), 'ysii_program', true);
        if ($slug) {
            printf('<meta name="ysii:program" content="%s">' . "\n", esc_attr($slug));
        }
    }
}, 1);

// 2) Shortcode widget rekomendasi: [ysii_reco]
add_shortcode('ysii_reco', function () {
    $s = ysii_settings();
    return sprintf(
        '<div id="ysii-reco" data-whatsapp="%s" data-daftar-url="%s"></div>',
        esc_attr($s['whatsapp']),
        esc_url($s['daftar_url'])
    );
});

// 3) Halaman pengaturan: Pengaturan > YSII Tracker
add_action('admin_menu', function () {
    add_options_page('YSII Tracker', 'YSII Tracker', 'manage_options', 'ysii-tracker', 'ysii_s
ettings_page');
});

add_action('admin_init', function () {
    register_setting('ysii_tracker', YSII_OPTION, [
        'sanitize_callback' => function ($in) {
            return [
                'api_url'      => esc_url_raw($in['api_url'] ?? ''),
                'privacy_url'  => esc_url_raw($in['privacy_url'] ?? ''),
                'consent'      => in_array($in['consent'] ?? '', ['required', 'implied'], tru
e) ? $in['consent'] : 'required',
                'mirror_forms' => in_array($in['mirror_forms'] ?? '', ['auto', 'marked', 'of
f'], true) ? $in['mirror_forms'] : 'auto',
                'whatsapp'     => preg_replace('/\D/', '', $in['whatsapp'] ?? ''),
                'daftar_url'   => esc_url_raw($in['daftar_url'] ?? ''),
            ];
        },
    ]);
});

function ysii_settings_page() {
    $s = ysii_settings();
    ?>
    <div class="wrap">
        <h1>YSII Tracker</h1>
        <form method="post" action="options.php">
            <?php settings_fields('ysii_tracker'); ?>
            <table class="form-table" role="presentation">
                <tr><th><label for="api_url">URL API</label></th>
                    <td><input class="regular-text" id="api_url" name="<?php echo YSII_OPTION;
?>[api_url]" value="<?php echo esc_attr($s['api_url']); ?>"></td></tr>
                <tr><th><label for="privacy_url">Halaman kebijakan privasi</label></th>
                    <td><input class="regular-text" id="privacy_url" name="<?php echo YSII_OPT
ION; ?>[privacy_url]" value="<?php echo esc_attr($s['privacy_url']); ?>"></td></tr>
                <tr><th>Persetujuan cookie</th>
                    <td><select name="<?php echo YSII_OPTION; ?>[consent]">
```

```
                        <option value="required" <?php selected($s['consent'], 'required'); ?>
>Tampilkan banner (disarankan, UU PDP)</option>
                        <option value="implied" <?php selected($s['consent'], 'implied'); ?>>T
anpa banner</option>
                    </select></td></tr>
                <tr><th>Salin form ke sistem lead</th>
                    <td><select name="<?php echo YSII_OPTION; ?>[mirror_forms]">
                        <option value="auto" <?php selected($s['mirror_forms'], 'auto'); ?>>Ot
omatis (form yang punya kolom HP/email)</option>
                        <option value="marked" <?php selected($s['mirror_forms'], 'marked'); ?
>Hanya form bertanda data-ysii-form</option>
                        <option value="off" <?php selected($s['mirror_forms'], 'off'); ?>>Mati
</option>
                    </select></td></tr>
                <tr><th><label for="whatsapp">No. WhatsApp admin</label></th>
                    <td><input id="whatsapp" name="<?php echo YSII_OPTION; ?>[whatsapp]" value
="<?php echo esc_attr($s['whatsapp']); ?>" placeholder="6281234567890"></td></tr>
                <tr><th><label for="daftar_url">URL halaman pendaftaran</label></th>
                    <td><input class="regular-text" id="daftar_url" name="<?php echo YSII_OPTI
ON; ?>[daftar_url]" value="<?php echo esc_attr($s['daftar_url']); ?>"></td></tr>
            </table>
            <?php submit_button(); ?>
        </form>
        <p>Halaman program: isi <em>custom field</em> <code>ysii_program</code> dengan slug pr
ogram (sama dengan di dashboard),
        atau gunakan URL berpola <code>/program/&lt;slug&gt;</code>. Tambahkan shortcode <code
>[ysii_reco]</code> untuk widget rekomendasi.</p>
    </div>
    <?php
}
```

backend/Dockerfile — Image kontainer aplikasi (15 baris)

```
FROM python:3.13-slim

ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
RUN useradd --create-home appuser && mkdir -p /app/models && chown -R appuser /app
USER appuser

EXPOSE 8000
# 1 worker: status pelatihan model & rate limit disimpan di memori proses
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--proxy-headers", "--f
orwarded-allow-ips", "*"]
```

docker-compose.yml — Orkestrasi layanan (69 baris)

```
services:
  db:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: ysii
      POSTGRES_USER: ysii
      POSTGRES_PASSWORD: ${DB_PASSWORD:?isi DB_PASSWORD di .env}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ysii -d ysii"]
      interval: 10s
      retries: 5

  api:
```

```
    build: ./backend
    restart: unless-stopped
    env_file: .env
    environment:
      DATABASE_URL: postgresql+psycopg://ysii:${DB_PASSWORD}@db:5432/ysii
      MODEL_DIR: /app/models
    volumes:
      - models:/app/models
      - ./secrets:/secrets:ro
    depends_on:
      db:
        condition: service_healthy

  scheduler:
    build: ./backend
    restart: unless-stopped
    command: python -m jobs.scheduler
    env_file: .env
    environment:
      DATABASE_URL: postgresql+psycopg://ysii:${DB_PASSWORD}@db:5432/ysii
      MODEL_DIR: /app/models
    volumes:
      - models:/app/models
      - ./secrets:/secrets:ro
    depends_on:
      db:
        condition: service_healthy

  nginx:
    image: nginx:1.27-alpine
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    environment:
      API_DOMAIN: ${API_DOMAIN:-api.ysii.org}
    volumes:
      - ./nginx/templates:/etc/nginx/templates:ro
      - ./nginx/certbot/conf:/etc/letsencrypt:ro
      - ./nginx/certbot/www:/var/www/certbot:ro
    depends_on:
      - api

  certbot:
    image: certbot/certbot:v2.11.0
    profiles: ["certbot"]
    volumes:
      - ./nginx/certbot/conf:/etc/letsencrypt
      - ./nginx/certbot/www:/var/www/certbot

volumes:
  pgdata:
  models:
```

nginx/templates/default.conf.template — Konfigurasi server web (24 baris)

```
# Template diproses oleh image nginx (envsubst): ${API_DOMAIN} diganti dari environment.

# Resolver DNS Docker: alamat container API di-resolve ulang setiap request,
# sehingga nginx tidak 502 setelah container API dibuat ulang (update/deploy).
resolver 127.0.0.11 valid=10s ipv6=off;
map $host $api_upstream { default api:8000; }

server {
    listen 80;
    server_name ${API_DOMAIN};

    location /.well-known/acme-challenge/ { root /var/www/certbot; }
```

```
13
14      # Sebelum sertifikat ada, layani HTTP saja (lihat README langkah HTTPS)
15      location / {
16          if (-f /etc/letsencrypt/live/${API_DOMAIN}/fullchain.pem) { return 301 https://$host$r
    equest_uri; }
17          proxy_pass http://$api_upstream;
18          proxy_set_header Host $host;
19          proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
20          proxy_set_header X-Forwarded-Proto $scheme;
21      }
22  }
23
24  include /etc/nginx/conf.d/https.inc*;
```